CONSTRUCTOR

Coustructor is a special non-static member which is used to load non-static members of a class into the object

The basic purpose of using the constructor is to create an object.

NOTE: i) The name of the constructor must be same as that of class name

ii) The constructor is just similar to the a method except for return type.

17/02/2022

iii) In java, every class created must and should have a constructor.

Therefore, the compiler will always check whether a programmer has created atleast one constructor inside the class or not.

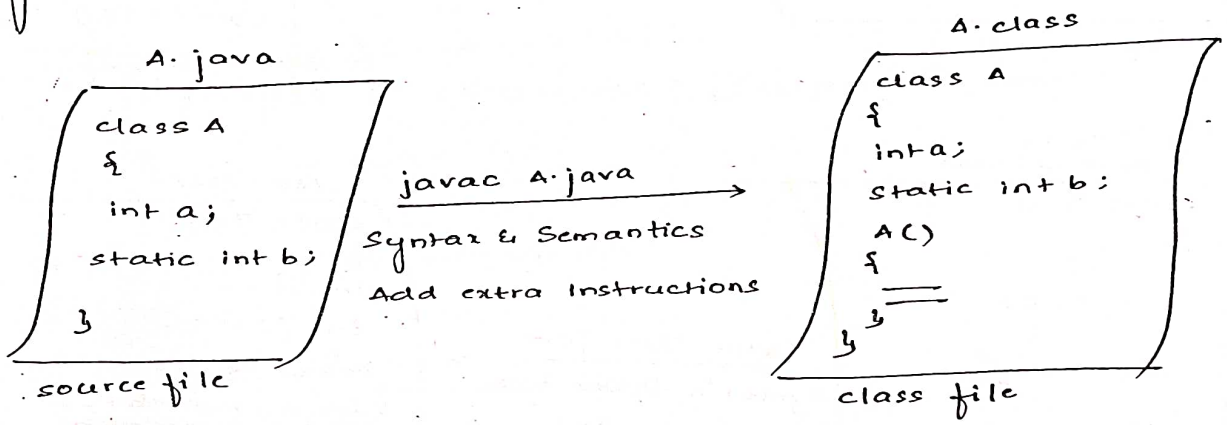
If it is not created, then compiler will add a constructor into the class file, ~~search~~ such constructors are known as DEFAULT CONSTRUCTORS.

▶ DEFAULT CONSTRUCTOR:

A No-Argument Constructor added by the compiler during the compilation is known as Defaault constructor.

It will be added only when the user forgets to create even.. a single constructor inside the class.

Eg:

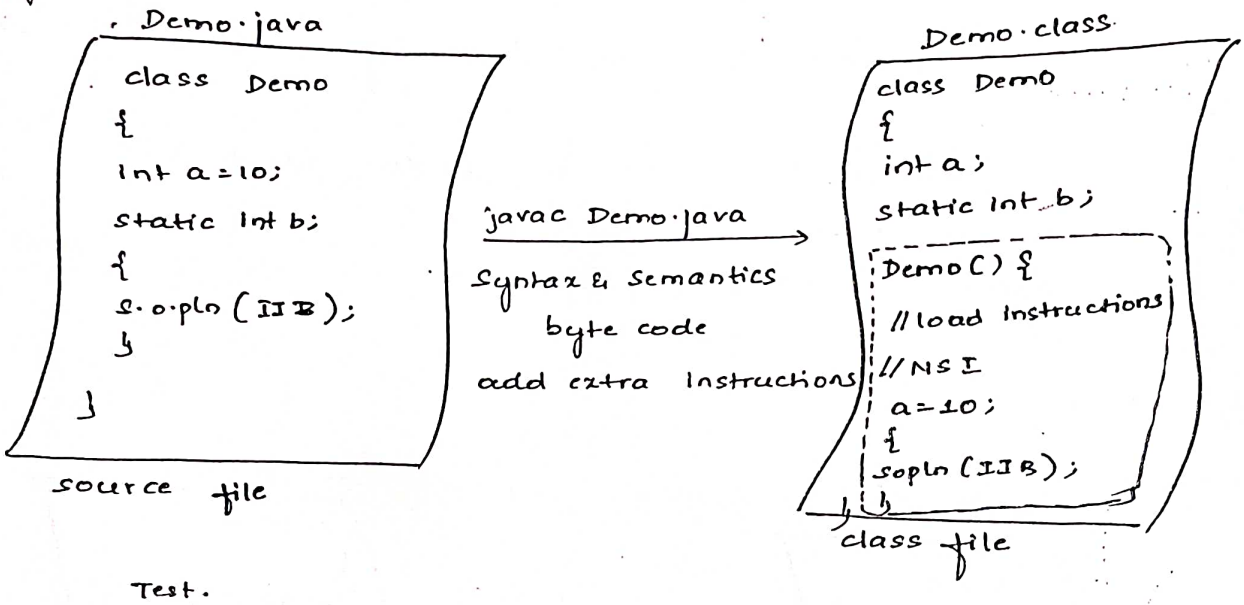


▶ PREDEFINED STATEMENTS INSIDE CONSTRUCTOR BODY:

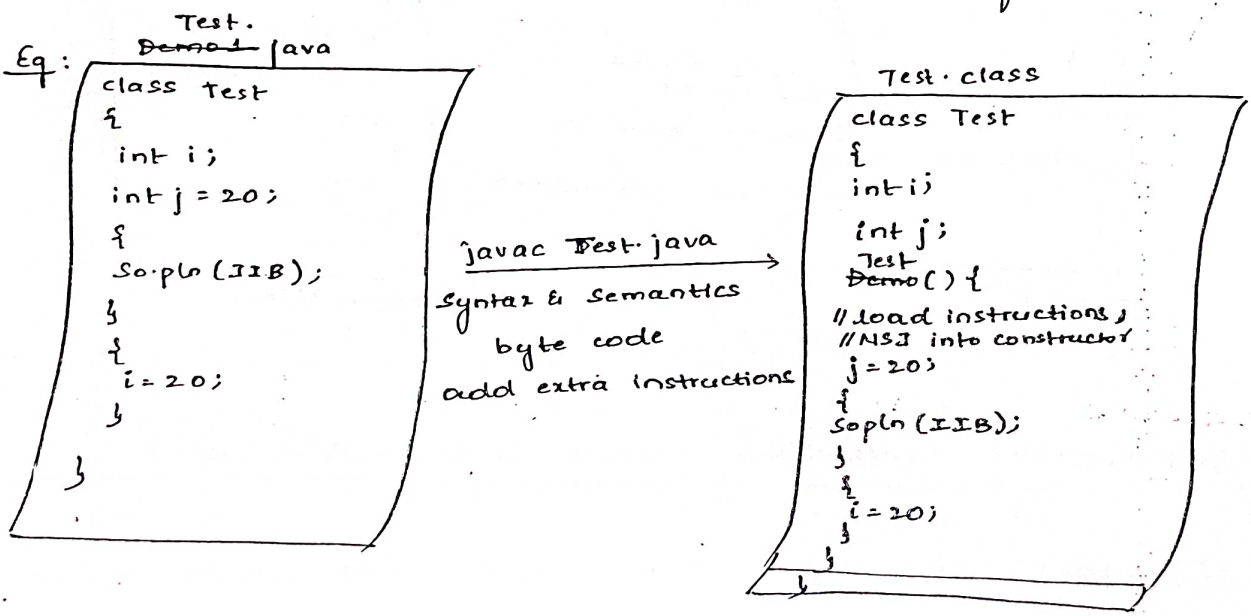
> compiler will add the instructions which helps us to load the non-static members of the class. (Non-static variables & Non static methods) into the object

Non-static initializers written by programmer will be loaded into side the constructor by the compiler

Eq:



Eq:



▶ USER - DEFINED CONSTRUCTORS :-

The constructors created by the users or progr -ers are known as user-defined constructors.

In java, the user defined constructors are clas into two types:

1) No-Argument constructor.

2) Parameterised constructor.

SYNTAX TO CREATE A CONSTRUCTOR :

```
[Modifier] Class Name ([Formal Arguments])
{
    // java statements;
}
```

▶ NO-ARGUMENTS CONSTRUCTOR:

A constructor which doesn't have a formal arguments declared is known as No-arguements constructor.

NOTE:

Inside class files, even the user defined constructors will have the pre-defined statements as its first instructions.

Eg:-

Demo1.java

```
class Demo1
{
  Demo1()
  {
  So.pln("I am a
  no-argument
  constructor");
  }
}
```

source file

javac Demo1.java

Syntax & Semantics

byte code.

Demo1.class

```
class Demo1
{
Demo1()
{
//load instructions
//NSI into constructor
//user-defined stmts
Sopln("I am a no-Arg
 constructor);
}
}
```

class file

Eg2:

Demo2.java

```
class Demo2
{
 int a=10;
Demo2()
 {
Sopln(I am a No-
 Arg constructor")
 }
{
 Sopln(IIB);
}
}
```

Source file

javac Demo2.java

Demo2.class

```
class Demo2
{
 int a;
 Demo2()
 {
//Load Instructions
//NSI
 a=10;
{
 Sopln(IIB);
}
Sopln("IAN-A.M);
}}
```

class file

LOADING PROCESS OF A OBJECT:

1) 'new' creates a block of memory inside heap are

2) The constructor used for object creation is called implicitly

3) Loading of Non-static members into the object takes place.

→ a) If any Non-static variable is present, it will be loaded with default value.

b) If any Non-static method is present, its refere will be stored inside the object.

4) If any Non-static initializers are present, they get executed directly.

5) If any user-written instructions are present, th get executed.

6) Once the execution of constructor is complete, th address of Object will be returned

NOTE:

Inside class files, even the user defined constructors will have the pre-defined statements as its first instructions.

Eg:-

Demo1.java

```
class Demo1
{
  Demo1()
  {
  So.pln("I am a
  no-argument
  constructor");
  }
}
```

Source file

javac Demo1.java

Syntax & Semantics

byte code.

Demo1.class

```
class Demo1
{
 Demo1()
 {
 //load instructions
 //NSI into constructor
 //user-defined stmts
 Sopln("I am a no-Arg
  constructor);
 }
}
```

class file

Eg2:

Demo2.java

```
class Demo2
{
 int a=10;
 Demo2()
 {
 Sopln(I am a No-
  Arg constructor")
 }
{
  Sopln(IIB);
}
}
```

Source file

javac Demo2.java

Demo2.class

```
class Demo2
{
 int a;
 Demo2()
 {
 //Load instructions
 //NSI
  a=10;
 {
  Sopln(IIB);
 }
 Sopln("IAN-A-M);
}}
```

class file

LOADING PROCESS OF A OBJECT:

1) 'new' creates a block of memory inside heap are

2) The constructor used for object creation is called implicitly

3) The Loading of Non-static members into the object takes place.

→ a) If any Non-static variable is present, it will be loaded with default value.

b) If any Non-static method is present, its refere will be stored inside the object.

4) If any Non-static initializers are present, they get executed directly.

5) If any user-written instructions are present, th get executed.

6) Once the execution of constructor is complete', th address of Object will be returned

NOTE:

Whenever we create an object using No-argument constructor, we need to initialize the states using extra instruction which is the main disadvantage of No-argument constructors. ie we will not be able to initialise the states of object during object creation itself.

To overcome this problem, java provides us one more member known as parameterised constructor.

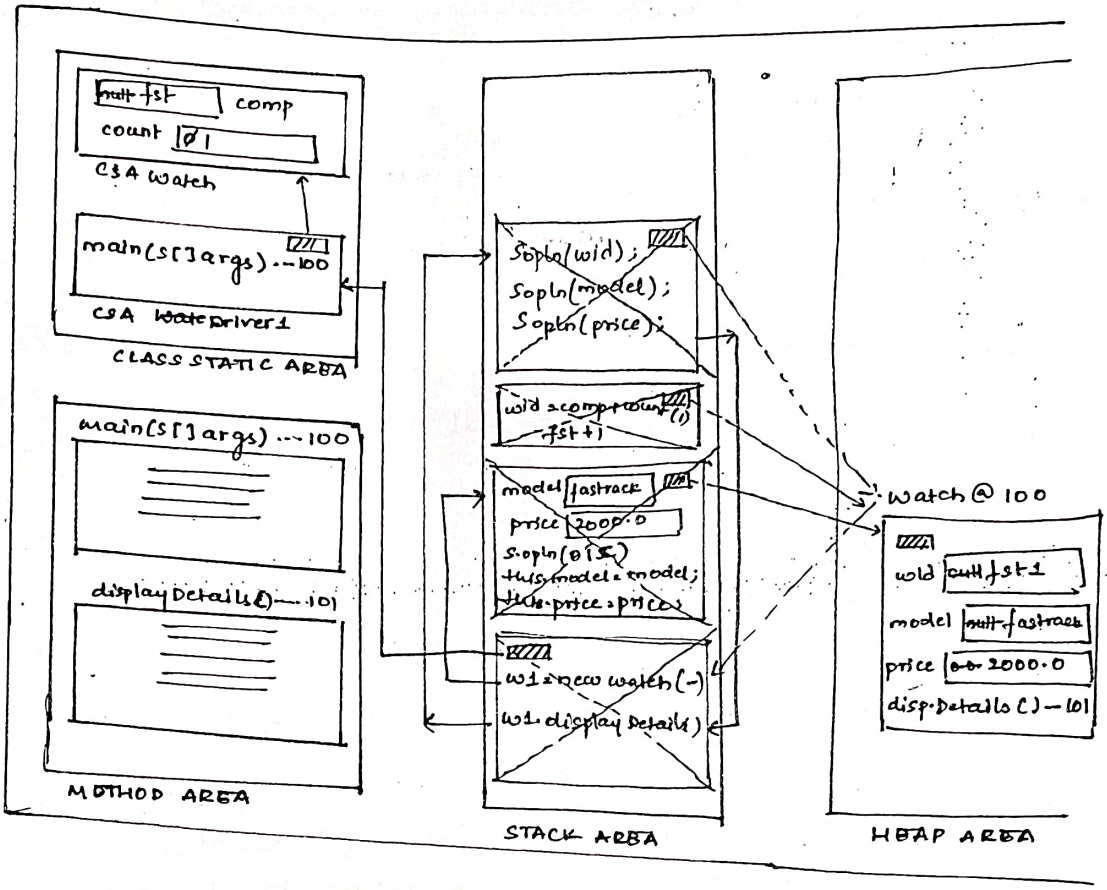
PARAMETERISED CONSTRUCTOR:-

A constructor which has formal arguments created during its declaration is known as parameterised constructor.

The parameterised constructors are used for initializing states of Object

Eg:

```
class Watch
{
String wid;
String model;
double price;
static string comp = "fst";
static int cocnt;
{
wid = comp + ++cocnt;
}
//constructor
Watch (String model, double price)
{
System.out.println("Object is created");
//assign the local variable values to global variables
this.model = model;
this.price = price;
}
public void displayDetails()
{
System.out.println("watch ID : " + wid);
System.out.println("Model : " + model);
System.out.println("Price : " + price);
}
}
class Driver1
{
public static void main(String[] args)
{
Watch w1 = new Watch("fasttrack", 2000);
w1.displayDetails();
}
}
```



Op: Object is created.

Watch Id : fst 1

model : fastrack

price : 2000.0

CONSTRUCTOR OVERLOADING:

* A class having more than one constructor is know as Constructor overloading
* We can have any number of constructors inside th class, but they have to follow certain rules.

1. The name of all the constructors should be same.
2. The declaration of the constructors ~~shold~~ should be differing either in length or in type or in order of declaration.

Eg:
```
class Lion
{
 int leg;
 String gender;
 String colour;
 double weight;

 Lion()
 {
 }
 Lion(double weight)
 {
   this.weight = weight;
 }
 Lion(String gender)
 {
   this.gender = gender;
 }
```

```
Lion (int leg, String gender, String colour, double weight)
{
  this.leg = leg;
  this.gender = gender;
  this.colour = colour;
  this.weight = weight;
}
}
class Driver2
{
  public static void displayDetails()
  {
  System.out.println("No. of legs :" + leg);
  System.out.println("Gender :" + gender);
  System.out.println("colour :" + colour);
  System.out.println("weight :" + weight);
  }
}
class Driver2
{
public static void main(String[] args)
 {
   Lion L1 = new Lion();
   L1.displayDetails();
   Lion L2 = new Lion("male");
   L2.displayDetails();
   Lion L3 = new Lion(4, "female", "brown", 230);
   L3.displayDetails();
 }
}
```